Tecnologia em Análise e Desenvolvimento de Sistemas - TADS

Estrutura de Dados

TADS / IFRS - 2026-1

Prof. ***Luciano Vargas Gonçalves***

E-mail: luciano.goncalves@riogrande.ifrs.edu.br

INSTITUTO FEDERAL
Rio Grande do Sul
Campus Rio Grande

Tecnologia em
ANÁLISE E
DESENVOLVIMENTO
DE SISTEMAS

Aula 2 – Ponteiros e Vetores

Relembrar a Memória RAM

- **Memória RAM**
  - Conjunto de ***Células*** de 1 Byte, composta de um endereço e um espaço para armazenar um *dado ou valor*;
  - **Célula de Memória (Endereço, Dados):**
    - **Endereço:** Os bytes são numerados sequencialmente. O número de identificação de um byte é o seu ***endereço (= address) escrito em hexadecimal.***
      - *Exemplo:*
        - *Endereço (e943af) de um Byte da memória*
    - ***Dado ou Valor (informação armazenada):*** *A memória RAM de qualquer computador é uma sequência de bytes (cada linha da tabela armazena um Byte (8 Bits de informação)*
      - *Ex: 1 Pente de 1GB ≃ 1bilhão de Bytes ou linhas na tabela.*

Endereço	Valor (Célula 1 Byte)
e943ab	
e943ac	
e943ad	
e943ae	
e943af	01011001
e943b0	
e943b1	
e943b2	

Memória RAM

Memória RAM

- ***Célula de Memória:***
  - ***Endereço de memória***
    - Identifica uma célula de memória;
    - O seu uso é controlado pelo Sistema Operacional (SO) dentro dos processos do sistema;
    - Cada processo cria uma tabela de endereços e informações.
  - ***Valor:***
    - Conteúdo ou informação armazenado na memória, o valor dado a alguma informação (dados, ou programas);

Endereço	Valor
e943ab	
e943ac	
e943ad	
e943ae	
e943af	
e943b0	
e943b1	
e943b2	

Memória RAM - Ex

Memória RAM

- Tipos de dados e Espaço ocupado na Memória RAM
  - Um char ocupa 1 byte (8bits).
  - Um int ocupa 4 bytes (32bits) em alguns computadores e 8 bytes em outros (sizeof (int)).
  - Um double ocupa usualmente 64 bits (sizeof (double)).
  - O endereço da variável (informação) é o endereço do primeiro byte alocado.

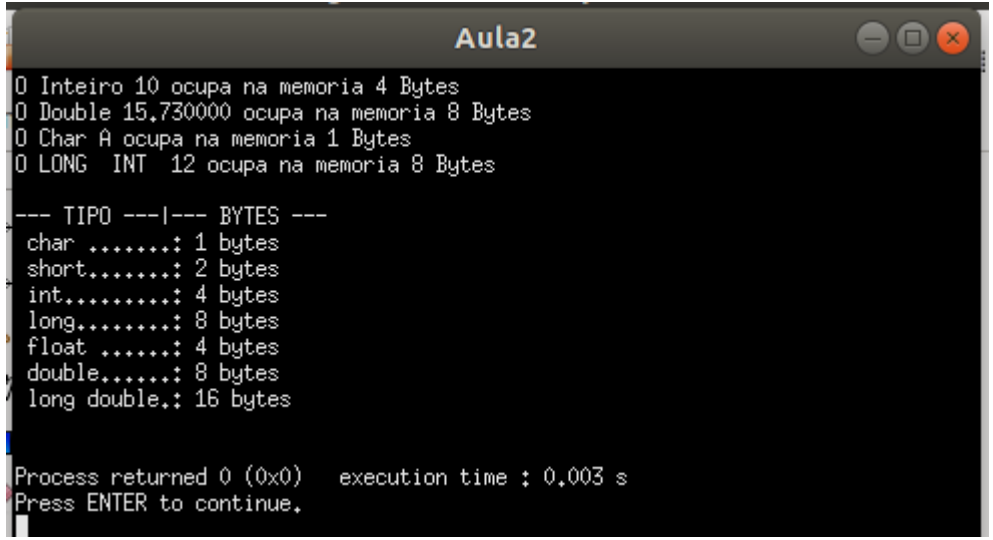
Tipo inteiro	Tamanho	Valor mínimo	Valor Máximo
signed char	8	−128	127
short int	16	−32.768	32.767
int	32	−2.147.483.648	2.147.483.647
long int	32	−2.147.483.648	2.147.483.647
long long int	64	−9.223.372.036.854.775.808	9.223.372.036.854.775.807
unsigned char	8	0	255
unsigned short int	16	0	65535
unsigned int	32	0	4.294.967.295
unsigned long int	32	0	4.294.967.295
unsigned long long int	64	0	18.446.744.073.709.551.615

Bits

Memória RAM

- Tipos de Dados - SIZEOF

```c
int v_inteiro = 10;
double v_double = 15.73;
char v_char = 'A';
long int v_longInt = 12;

printf("O Inteiro %d ocupa na memoria %d Bytes \n",v_inteiro,sizeof(v_inteiro));
printf("O Double %f ocupa na memoria %d Bytes \n",v_double,sizeof(v_double));
printf("O Char %c ocupa na memoria %d Bytes \n",v_char,sizeof(v_char));
printf("O LONG  INT  %d ocupa na memoria %d Bytes \n",v_longInt,sizeof(v_longInt));
```



	Endereço	Valor	Tipo de dado
4 bytes	e943a9	10	v_inteiro
8 bytes	e943ad	15.73	v_double
	e943b4		
1 bytes	e943b5	'A'	v_char
	e943b6		

Memória RAM

- ***Operador '&', copia o endereço de uma posição de memória ocupada.***
  - Operador '&' consulta ou copia o ***endereço do primeiro Byte de uma variável*** que se encontra na memória;

```
#include <stdio.h>
int main()
{
    system("clear");
    int x = 10;
    double y = 20, z = 30;

    printf("\nEndereco de X = %x e o valor X= %d \n",&x,x);
    printf("\nEndereco de Y = %x e o valor Y= %f \n",&y,y);
    printf("\nEndereco de Z = %x e o valor Z= %f \n",&z,z);

    exit(0);
}
```

consulta

EndereçoY	Valor	Variável
e943a0	10	x
e943ad	20	y
e943b5	30	z
e943b6		
e943b7		
e943b8		

Memória RAM

- ***Operador '&'***

```c
#include <stdio.h>
int main()
{
    system("clear");
    int x = 10;
    double y = 20, z = 30;

    printf("\nEndereco de X = %x e o valor X= %d \n",&x,x);
    printf("\nEndereco de Y = %x e o valor Y= %f \n",&y,y);
    printf("\nEndereco de Z = %x e o valor Z= %f \n",&z,z);

    exit(0);
}
```

```
Endereco de X = 4d984464 e o valor X= 10

Endereco de Y = 4d984468 e o valor Y= 20.000000

Endereco de Z = 4d984470 e o valor Z= 30.000000
```

Saída no terminal

Operadores '%x' ou '%p' imprimem um valor, endereço, um Hexadecimal;

Ponteiros em C

- **Ponteiros**
  - ***São variáveis capazes de armazenar <u>APENAS</u> endereços de memória (hexadecimal);***
  - Os conceitos de endereço de memória e ponteiro são fundamentais em qualquer linguagem de programação, embora fiquem ocultos em algumas delas.
  - Em C, esses conceitos são explícitos. O conceito de ponteiro não é fácil e, é preciso fazer algum esforço para dominá-lo.
  - ***A partir do endereço da variável, pode-se acessar o conteúdo desta posição de memória, deforma <u>"indiretamente ou referenciada"</u>, apenas com o endereço da posição;***
    - ***Pode-se também armazenar o endereço de outra posição de memória;***

Ponteiro

- ***Definição***:
  - ***Ponteiro (*Px) é uma variável com a capacidade de armazenar um endereço de memória (hexadecimal);***
  - Na declaração do ponteiro usa-se o **'*'** para indicar um ponteiro:
  - O ponteiro deve ter *o mesmo Tipo* da variável referenciada;
    - Ex: Variável ***inteira*** "x", terá um ponteiro inteiro *px;
      - Exemplo:
        - ***px = &x;***
        - Ponteiro *px recebe e armazena o endereço da variável "x".
        - ***Agora *px é uma referência para X;***
        - ****px = x; são equivalentes;***

Endereço	Valor	Variável
e943ab	10	x
e943ac		
e943ad		
e943ae		
e943af	e943ab	*px
e943b0		
e943b1		
e943b2		

Ponteiro

Ponteiro

- Exemplo: ***X e a referência *px***

```
int main()
{
    printf("\n Aula de Ponteiros!! \n");

    int x, *px;  //x variável e px o ponteiro

    x = 10;
    px = &x;

    printf(" Valor x = %d e o endereço de X = %x \n",x,&x);
    printf(" Valor px = endereço de %x \n",px);

}
```

```
Aula de Ponteiros!!
Valor X = 10 e o endereço de X = 18ac57ec
Valor px = endereço de 18ac57ec
```

Saída no Terminal

Ponteiro

Endereço	Valor	Variável
18ac57ec	10	X
....		
18ac57f0		
18ac57f1		
18ac57f2	18ac57ec	*px
18ac57f3		

Ponteiro – Valor referenciado

- O valor referenciado por ***px = x;***
  -

```
printf(" Conteúdo de X = %d e o conteúdo
        referenciado por px = %d\n",x,*px);
```

Saída no Terminal

```
Aula de Ponteiros!!
Valor X = 10 e o endereço de X = fb75aa6c
Valor px = endereço de fb75aa6c
Conteúdo de X = 10 e o conteúdo referenciado por px = 10
```

Mesmo Valor X = *PX = 10

Endereço	Valor	Variável
18ac57ec	10	x
....		
18ac57f0		
18ac57f1		
18ac57f2	18ac57ec	*px
18ac57f3		

Referência

Ponteiro - Operadores

- ***Operadores '*', '&', '%x' ou '%p'***
  - Operador **'*'** usado para declarar o ponteiro, e também para **consultar e/ou alterar o conteúdo do endereço referenciado;**
  - Operador ***'&'*** consulta e/ou copia o endereço de uma variável;
  - Operador '***%x***' ou '***%p***' imprimem um endereço;
    - Exemplo:
      - Valor armazenado em X = 10;
      - Endereço de &X = e943ab
      - Valor armazenado em Px = e943ab
      - Valor referenciado em *Px = 10
      - Endereço de &Px = e943af

Endereço	Valor	Variável
e943ab	10	x
e943ac		
e943ad		Ponteiro
e943ae		
e943af	e943ab	*px
e943b0		
e943b1		
e943b2		

Ponteiro - Equivalência

- O valor referenciado por *px = x e *py = y;

```
int x, *px;  //x variável e px o ponteiro
int y, *py;  //y é uma variavel e py e'o ponteiro

x = 10;   px = &x;
y = 20;   py = &y;

printf(" Valor X = %d e o endereço de X = %x \n",x,px);
printf(" Valor Y = %d e o endereço de Y = %x \n",y,py);

printf(" Valor referenciado px = %d",*px);
printf(" Valor referenciado py = %d\n",*py);
```

//Atribuição de Valores

```
Aula de Ponteiros!!
Valor X = 10 e o endereço de X = c8777ec0
Valor Y = 20 e o endereço de Y = c8777ec4
Valor referenciado px = 10 Valor referenciado py = 20
```

Saída no Terminal

Ponteiro

- Troca de endereços entre px = py; logo ***px = y e;***

```
printf("\n Aula de Ponteiros!! \n");

int x, *px;  //x variável e px o ponteiro
int y, *py;  //y é uma variavel e py e'o ponteiro

x = 10;   px = &x;
y = 20;   py = &y;

printf(" Valor X = %d e o endereço de X = %x \n",x,px);
printf(" Valor Y = %d e o endereço de Y = %x \n",y,py);

printf(" Valor referenciado px = %d",*px);
printf(" Valor referenciado py = %d\n",*py);

px = py;  //copia do endereço de py para px

printf(" Valor referenciado px = %d",*px);
printf(" Valor referenciado py = %d\n",*py);
```

px = x

```
Aula de Ponteiros!!
Valor X = 10 e o endereço de X = aab85ac0
Valor Y = 20 e o endereço de Y = aab85ac4
Valor referenciado px = 10 Valor referenciado py = 20
Valor referenciado px = 20 Valor referenciado py = 20
```

px = y

Saída no Terminal

Ponteiros

- Troca de conteúdo de *px = *py; logo ***x = y;***

```
x = 10;    px = &x;
y = 20;    py = &y;

printf(" Valor referenciado px = %d",*px);
printf(" Valor referenciado py = %d\n",*py);

*px = *py;

printf(" Valor x = %d e y = %d\n",x,y);
```

Variáveis x = y

```
Aula de Ponteiros!!
Valor referenciado px = 10 Valor referenciado py = 20
Valor x = 20 e y = 20
```

Saída Terminal

Troca entre X e Y

- Troca entre os valores x e y, através dos ponteiros *px, *py

```
Aula de Ponteiros!!

Valor x = 10 y = 20
Valor x = 20 y = 10
```

Saída Terminal

```c
int main()
{
    printf("\n Aula de Ponteiros!! \n");
    int x, y, *px, *py, aux;
    x = 10;   y = 20;
    px = &x;  py = &y;

    printf ("\n Valor x = %d y = %d",x,y);
    aux = *px;
    *px = *py;
    *py = aux;
    printf ("\n Valor x = %d y = %d\n",x,y);
}
```

Exercícios

1) Crie um programa em C, para ler duas variáveis inteira (A,B), referente a idade de duas pessoas, após crie uma variável S e um ponteiro para *ps. Armazene a soma dos valores A e B em S, com o uso do ponteiro *ps, Após:

    1) Mostre o resultado armazenado em S.
    2) Mostre o endereço armazenado pelo ponteiro **ps* e o endereço de S;
    3) Mostre o endereço do ponteiro **ps*, mostre o conteúdo referenciado por *ps;
    4) Defina um ponteiro *pab, mostre o endereço e o conteúdo da variável A, usando *pab;
    5) Após, troque o endereço armazenado em *pab, pelo endereço da variável B, mostre o seu conteúdo e endereço armazenado em pab;

Ponteiros

- Um ponteiro pode ter um valor especial - *NULL (nada)*
  - O ponteiro ***p = Null***, indica que o ponteiro inicia sem o apontamento para uma posição de memória;

```
{
    printf("\n Aula de Ponteiros!! \n");
    int *px = NULL;

    printf ("\n Valor px = %x \n",px);
}
```

```
Aula de Ponteiros!!

Valor px = (nil)
```

Endereço	Valor	Variável
e943ab		
e943ac		
e943ad		
e943ae		
e943af	NULL	*px
e943b0		
e943b1		
e943b2		

Alocação dinâmica

- **Alocação de memória direta “malloc”**
  - Comando reserva uma quantidade de memória pré-determinada e retorna o endereço do primeiro Byte alocado. Faz-se necessário CASTING.
  - Precisa definir o tamanho de memória a ser ocupada (“sizeof”)
  - Atenção - Espaço de memória sem uma variável associada,

```
int *px;     //declaração
px = (int*)malloc(sizeof(int));//alocar memória
*px = 20;
printf("\nEndereco de x = %p e o valor X=%d \n",px,*px);
free(px); //desalocar memória
exit(0);
```

Casting para o tipo Inteiro

Endereço	Valor	Variável
e943ab	20	???
e943ac		
e943ad		
e943ae		
e943af	e943ab	px
e943b0		
e943b1		
e943b2		

Ponteiros

- Alocação de memória “malloc”
  - Não existe uma variável associada a posição de memória;
  - Necessita validar a alocação de memória (IF);

```
int *px;     //declaração
px = (int*)malloc(sizeof(int));//alocar memória

if(px == NULL){ //validar alocação
    printf("Erro - Memória Insuficiente.\n");
    exit(0);
}
*px = 20;
printf("\nEndereco de x = %p e o valor X=%d \n",px,*px);
free(px); //desalocar memória
exit(0);
```

Reserva

Teste

Liberação

Endereço	Valor	Variável
e943ab	20	??
e943ac		
e943ad		Ponteiro
e943ae		
e943af	e943ab	px
e943b0		
e943b1		
e943b2		

Ponteiro direto, sem variável

Comando “Free” libera a memória;
--- Analogia com ir ao restaurante com reserva marcada;

Ponteiros em C

- O segundo argumento da função ***Scanf*** é o endereço da posição na memória, onde devem ser depositados as informações lidas do dispositivo entrada padrão:

```
int i;
scanf ("%d", &i);
```

Endereço onde será armazenada a informação

Exercícios

2) Crie um programa em C, para ler duas variáveis inteira (A,B), referente a idade de duas pessoas. Crie um ponteiro para *ps. Armazene a soma dos valores A e B em um espaço de memória alocado dinamicamente (use Malloc) e referenciada pelo ponteiro *ps, Após:

    1) Mostre o resultado de A+B, armazenado espaço referenciado por *ps;
    2) Mostre o endereço armazenado no ponteiro *Ps;
    3) Mostre o endereço do ponteiro *Ps;
    4) Mostre o endereço das variáveis A e B;

Exercícios

3) Crie um programa em C, para ler duas variáveis double (A,B), referente ao peso de duas pessoas;

   Aloque uma porção de memória do tipo double, atribua o endereço da memória alocada ao ponteiro *Ps, para armazenar a soma dos valores A e B;

   1) Mostre o resultado de A + B, armazenado espaço referenciado por *ps;
   2) Mostre o endereço armazenado no ponteiro *Ps, e o valor referenciado;
   3) Crie uma variável double C, e copie o valor referenciado no ponteiro *Ps para a variável C. Mostre o valor de C;

Exercícios

4) Com base no exercício anterior (3), copie o endereço armazenado no ponteiro *ps, para o ponteiro *qs;
   1) Some 100 ao valor referenciado no ponteiro *qs;
   2) Mostre os valores referenciados por *ps e *qs;
   3) Copie o valor do ponteiro *qs para a variável A;
   4) Atribua o endereço da variável B, ao ponteiro *qs;
   5) Use o ponteiro *qs, para subtrair 10 do valor da variável B;
   6) Mostre os valores de A, B e *qs.

Passagem por Valor

- Troca da valores entre duas variáveis x,y, usando função;

```
void troca1 (int a, int b) {
    int temp;
    temp = a;
    a = b;
    b = temp;
}

int main()
{
    printf("\n Aula de Ponteiros!! \n");
    int x = 10, y = 20;

    //passagem por valor
    printf("\n Troca 1 - Valor de X=%d e Y=%d",x,y);
    troca1(x,y);
    printf("\n Troca 1 - Valor de X=%d e Y=%d",x,y);
```

```
Valor X=10 e Y=20

Valor X=10 e Y=20
```

Troca não realizada

Passagem por Valor

- Troca dos valores x,y armazenados;

```c
void troca1 (int a, int b) {
    int temp;
    temp = a;
    a = b;
    b = temp;
}

int main()
{
    printf("\n Aula de Ponteiros!! \n");
    int x = 10, y = 20;

    //passagem por valor
    printf("\n Troca 1 - Valor de X=%d e Y=%d",x,y);
    troca1(x,y);
    printf("\n Troca 1 - Valor de X=%d e Y=%d",x,y);
```

```
Valor X=10 e Y=20

Valor X=10 e Y=20
```

Endereço	Valor	Variável
e943ab	10	x
e943ac		
e943ad	20	y
e943ae		
e943af	10	a
e943b0	20	b
e943b1		
e943b2		

Troca não realizada
X e Y, são diferentes A e B

Passagem por Valor

- Troca dos valores x,y armazenados;

```c
void troca1 (int a, int b) {
    int temp;
    temp = a;
    a = b;
    b = temp;
}

int main()
{
    printf("\n Aula de Ponteiros!! \n");
    int x = 10, y = 20;

    //passagem por valor
    printf("\n Troca 1 - Valor de X=%d e Y=%d",x,y);
    troca1(x,y);
    printf("\n Troca 1 - Valor de X=%d e Y=%d",x,y);
```

Troca na função, duplicidade de informação

```
Valor X=10 e Y=20

Valor X=10 e Y=20
```

Endereço	Valor	Variável
e943ab	10	x
e943ac		
e943ad	20	y
e943ae		
e943af	20	a
e943b0	10	b
e943b1		
e943b2		

Troca não realizada no main

Passagem por Referência - Ponteiros

- Troca dos valores de x,y, com o uso de ponteiros;

```
void troca2 (int *pa, int *pb) {
    int temp;
    temp = *pa;
    *pa = *pb;
    *pb = temp;
}


int main()
{
    printf("\n Aula de Ponteiros!! \n");
    int x, y;
    //passagem por referencia
    x = 10, y = 20;
    printf("\n Troca 2 - Valor de X=%d e Y=%d",x,y);
    troca2(&x,&y);
    printf("\n Troca 2 - Valor de X=%d e Y=%d",x,y);

}
```

Ponteiros *pa e *pb

Passagem por Referência

```
Valor X=10 e Y=20

Valor X=20 e Y=10
```

Troca realizada

Passagem por Referência

- Troca dos valores de x,y, com o uso de ponteiros;

```c
void troca2 (int *pa, int *pb) {
    int temp;
    temp = *pa;
    *pa = *pb;
    *pb = temp;
}


int main()
{
    printf("\n Aula de Ponteiros!! \n");
    int x, y;
    //passagem por referencia
    x = 10, y = 20;
    printf("\n Troca 2 - Valor de X=%d e Y=%d",x,y);
    troca2(&x,&y);
    printf("\n Troca 2 - Valor de X=%d e Y=%d",x,y);

}
```

Troca de informações é por meio de ponteiros

Endereço	Valor	Variável
e943ab	10	X
e943ac		
e943ad	20	y
e943ae		
e943af	e943ab	pa
e943b0	e943ad	pb
e943b1		
e943b2		

*pa = X e *pb = Y

Ponteiros e Passagem por Referência

- Troca da valores entre duas variáveis x,y;

```c
void troca2 (int *pa, int *pb) {
    int temp;
    temp = *pa;
    *pa = *pb;
    *pb = temp;
}


int main()
{
    printf("\n Aula de Ponteiros!! \n");
    int x, y;
    //passagem por referencia
    x = 10, y = 20;
    printf("\n Troca 2 - Valor de X=%d e Y=%d",x,y);
    troca2(&x,&y);
    printf("\n Troca 2 - Valor de X=%d e Y=%d",x,y);

}
```

Endereço	Valor	Variável
e943ab	20	X
e943ac		
e943ad	10	y
e943ae		
e943af	e943ab	pa
e943b0	e943ad	pb
e943b1		
e943b2		

```
Aula de Ponteiros!!

Troca 2 - Valor de X=10 e Y=20
Troca 2 - Valor de X=20 e Y=10
```

Saída terminal

Exercícios

4) Crie uma função somaAB para receber dois valores inteiros (A e B) e uma referência para inteiro (*pab). O ponteiro *pab retornará o resultado da soma de A e B;

```
void somaAB(int a, int b, int *pab){
```

   1) Na função main(), crie uma variável S que irá armazenar o valor da soma A e B, após mostre os valores A, B e S;

5) Crie uma função para receber duas referências para inteiros (*pa,*pb), divida o valor *pa pelo valor de *pb, atribua o resultado a variável double *pab;

```
void dividaAB(int *pa, int *pb, double *pab)
```

   1) Crie um programa (main) para usar as funções dividaAB;
   2) Leia dois valores A, B e calcule e mostre a divisão dos valores função soma(ex 4);
   3) Dívida os valores A e B e apresente o resultado, utilize a função item 2;

Vetores e Strings

- Vetores e Strings são ponteiros em C:
  - Um vetor e uma porção de memória contiguá de *mesmo tipo e o endereço do primeiro elemento é o mesmo do vetor*; Os demais elementos estão equidistantes em função do seu tamanho (memória ocupada);
  - Exemplo

```
int v [5] = {3,4,5,2,1};
```

Endereço	Valor	Variável
e943ab		
e943ac	3	
e943b0	4	
e943b4	5	
e943b8	2	
e943bc	1	
e943bf		
e943c0	e943ac	v

Execução

Vetores e Strings

```c
void imprimeVetor(int vt[], int n){
    for (int i=0;i<n;i++)
        printf("\n v[%d] = %d ",i,vt[i]);
    printf("\n");
}
```

Função sem ponteiro

```c
void imprimeVetorPonteiro(int *p, int n){
    for (int i=0;i<n;i++)
        printf("\n v[%d] = %d ",i,*(p+i));
    printf("\n");
}
```

Função com ponteiro

ponteiro + incremento

```c
void somaDez(int *p, int n){…
```

```c
int main()
{
    int v [5] = {3,4,5,2,1};
    int n = 5;

    imprimeVetor (v,n); //passagem por referencia
    imprimeVetorPonteiro (v,n); //passagem por referencia
```

Vetor não necessita “&”

```
v[0] = 3
v[1] = 4
v[2] = 5
v[3] = 2
v[4] = 1

v[0] = 3
v[1] = 4
v[2] = 5
v[3] = 2
v[4] = 1
```

Execução

Endereço	Valor	Variável
e943ab		
e943ac	3	v[0]
e943b0	4	v[1]
e943b4	5	v[2]
e943b8	2	v[3]
e943bc	1	v[4]
e943bf	e943ac	Vt
e943c0	e943ac	V
e943c1	e943ac	p

Vetores e Strings

```
void imprimeVetor(int vt[], int n){…

void imprimeVetorPonteiro(int *p, int n){…

void somaDez(int *p, int n){
    for (int i=0;i<n;i++)
        *(p+i)+=10;
}

int main()
{
    int v [5] = {3,4,5,2,1};
    int n = 5;

    imprimeVetor (v,n); //passagem por referencia
    imprimeVetorPonteiro (v,n); //passagem por referencia
    somaDez(v,n);
    imprimeVetor (v,n); //passagem por refencia
```

Função acrescenta 10 a cada elemento de V , *(p+i)

Vetor não necessita “&”

```
v[0] = 3
v[1] = 4
v[2] = 5
v[3] = 2
v[4] = 1

v[0] = 3
v[1] = 4
v[2] = 5
v[3] = 2
v[4] = 1

v[0] = 13
v[1] = 14
v[2] = 15
v[3] = 12
v[4] = 11
```

Execução

Vetores e Strings

```c
#include <stdio.h>
void imprimeString(char st[], int n){
    for (int i=0;i<n;i++)
        printf("\n S[%d] = %c ",i,st[i]);
    printf("\n");
}
void imprimeStringPonteiro(char *p, int n){
    for (int i=0;i<n;i++)
        printf("\n S[%d] = %c ",i,*(p+i));
    printf("\n");
}
void codigoAsc(char *p, int n){
    for (int i=0;i<n;i++)
        printf("\n%c = %d",*(p+i),*(p+i));
}
int main()
{
    char v [] = "Estrutura de Dados";
    int n = sizeof(v);
    imprimeString (v,n); //passagem por referencia
    imprimeStringPonteiro (v,n); //passagem por refencia
    codigoAsc(v,n);
    exit(0);
}
```

Função com Vetor de Char;

Função ponteiro de Char;

Vetores e Strings

```c
#include <stdio.h>
void imprimeString(char st[], int n){
    for (int i=0;i<n;i++)
        printf("\n S[%d] = %c ",i,st[i]);
    printf("\n");
}
void imprimeStringPonteiro(char *p, int n){
    for (int i=0;i<n;i++)
        printf("\n S[%d] = %c ",i,*(p+i));
    printf("\n");
}
void codigoAsc(char *p, int n){
    for (int i=0;i<n;i++)
        printf("\n%c = %d",*(p+i),*(p+i));
}
int main()
{
    char v [] = "Estrutura de Dados";
    int n = sizeof(v);
    imprimeString (v,n); //passagem por referencia
    imprimeStringPonteiro (v,n); //passagem por refencia
    codigoAsc(v,n);
    exit(0);
}
```

Saídas do programa

```
S[0] = E
S[1] = s
S[2] = t
S[3] = r
S[4] = u
S[5] = t
S[6] = u
S[7] = r
S[8] = a
S[9] =
S[10] = d
S[11] = e
S[12] =
S[13] = D
S[14] = a
S[15] = d
S[16] = o
S[17] = s
S[18] =
```

imprimeString()

```
S[0] = E
S[1] = s
S[2] = t
S[3] = r
S[4] = u
S[5] = t
S[6] = u
S[7] = r
S[8] = a
S[9] =
S[10] = d
S[11] = e
S[12] =
S[13] = D
S[14] = a
S[15] = d
S[16] = o
S[17] = s
S[18] =
```

imprimeStringPonteiro()

```
E = 69
s = 115
t = 116
r = 114
u = 117
t = 116
u = 117
r = 114
a = 97
  = 32
d = 100
e = 101
  = 32
D = 68
a = 97
d = 100
o = 111
s = 115
```

codigoAsc()

Exercícios

6) Escreva uma função ***menorVetor*** que receba um vetor inteiro ***v[0..n-1],*** a quantidade de elementos do vetor(n), o endereço de uma variável inteira ***menor_valor*** (passagem por referência), e deposite nesta variável o valor do menor elemento do vetor.

```
void menorVetor(int *vt, int n, int *menor){
```

7) Escreva uma função ***maiorVetor*** que receba um vetor inteiro ***v[0..n-1],*** a quantidade de elementos do vetor(n), o endereço de uma variável inteira **maior_*valor***(passagem por referência), e deposite nesta variável o valor do **maior** elemento do vetor.

```
void maiorVetor(int *vt, int n, int *maior){
```

8) Escreva uma função ***menorMaiorVetor*** que receba um vetor inteiro ***v[0..n-1],*** a quantidade de elementos do vetor(n), e o endereço de duas variável inteira ***menor e maior*** (passagem por referência), e deposite nestas variáveis o valor do menor e maior elemento do vetor. Utilize as funções ***menorVetor e maiorVetor;***

```
void menorMaiorVetor(int *vt, int n, int *menor, int *maior){
```

Exercícios

9) Escreva uma função ***menorVetorPonteiro*** que receba um vetor inteiro ***v[0..n-1], e*** a quantidade de elementos do vetor(n), a função retorna ***o endereço do menor*** elemento do vetor.

```
int* menorVetorPonteiro(int *vt, int n);
```

10) Escreva uma função ***maiorVetorPonteiro*** que receba um vetor inteiro ***v[0..n-1], e*** a quantidade de elementos do vetor(n), a função retorna o ***endereço do maior*** elemento do vetor.

```
int* maiorVetorPonteiro(int *vt, int n);
```

11) Escreva uma função ***menorMaiorVetorPonteiro*** que receba um vetor inteiro ***v[0..n-1],*** a quantidade de elementos do vetor(n), e o endereço de duas variável inteira ***menor e maior*** (passagem por referência), e deposite nessas variáveis o valor do menor e maior elemento do vetor. Utilize as funções ***menorVetor e maiorVetor;***

```
void menorMaiorVetorPonteiros(int *vt, int n, int *menor, int *maior);
```

Exercícios

12) Escreva uma função ***Converte*** que converta minutos em horas-e-minutos.

- A função recebe um inteiro ***mnts*** e os endereços de duas variáveis inteiras, digamos **h** e **m**, e atribui valores a essas variáveis, de modo que ***m*** seja o resto da divisão de mnts%60 ***e que h seja*** igual a (***mnts/60) inteiro***.
- Escreva também uma função main que use a função ***Converte***.

Exercícios

13)Escreva um função para receber uma String(vetor de char), a quantidade elementos do vetor e um ponteiro (vetor) para armazenar todas as vogais presentes na String.

14)Escreva um função para receber uma String(vetor de char), a quantidade elementos do vetor e um ponteiro (vetor) para armazenar todas as consoantes presentes na String.

15)Escreva um função para receber uma String(vetor de char), e a quantidade elementos do vetor e retorne a quantidade de vogais na string.

16)Escreva um função para receber uma String(vetor de char), e a quantidade elementos do vetor e retorne a quantidade de consoantes na string.

- Crie um programa para utilizar as funções acima.

Ponteiros em C

- Há vários tipos de ponteiros:
  - Ponteiros para caracteres, para inteiros, para registros etc.
  - O computador precisa saber de que tipo de ponteiro você está falando.
  - Para declarar um ponteiro “*p” para um inteiro, diga
    - int *p;
  - Um ponteiro de ponteiro “**r” um ponteiro que apontará para outro ponteiro;
    - Ponteiro de Ponteiro int **r;

Ponteiros de Ponteiro **

```c
int x = 10, y = 20;
int *px, *py, **ppx;

px = &x;
py = &y;
ppx = &px;

printf("\n Valor de X = %d",x);
printf("\n Valor de *PX = %d",*px);
printf("\n Valor de *PPX = %d",**ppx);

printf("\n Endereco de X = %p",&x);
printf("\n Endereco de *PX = %p",&px);
printf("\n Endereco de *PPX = %p",&ppx);
```

- Ponteiro de Ponteiro (**ppx)

```
Valor de X = 10
Valor de *PX = 10
Valor de *PPX = 10
Endereco de X = 0x7ffe34e9ca88
Endereco de *PX = 0x7ffe34e9ca90
Endereco de *PPX = 0x7ffe34e9ca98
Conteudo de *PX = 0x7ffe34e9ca88
Conteudo de *PPX = 0x7ffe34e9ca90
Conteudo referenciado *PX = 10
Conteudo referenciado *PPX = 10
```

Ponteiro de Ponteiro (*ppx) em C

```c
int x = 10;
int *px, **ppx;

px = &x;
ppx = &px;

printf("\n Valor de X = %d",x);
printf("\n Valor de *PX = %d",*px);
printf("\n Valor de *PPX = %d",**ppx);

printf("\n Endereco de X = %p",&x);
printf("\n Endereco de *PX = %p",&px);
printf("\n Endereco de *PPX = %p",&ppx);

printf("\n Conteudo de *PX = %p",px);
printf("\n Conteudo de *PPX = %p",ppx);

printf("\n Conteudo referenciado *PX = %d",*px);
printf("\n Conteudo referenciado *PPX = %d\n",**ppx);
```

Ponteiro (ppx) → Ponteiro (px) → variável (x)
**ppx → *px → x

Endereço	Valor	Variável
e943ab	10	x
e943ac		
e943ad	20	y
e943ae		
e943af	e943ab	px
e943b0	e943ad	py
e943b1	e943af	ppx
e943b2		

Exemplo12-ponteiro.c

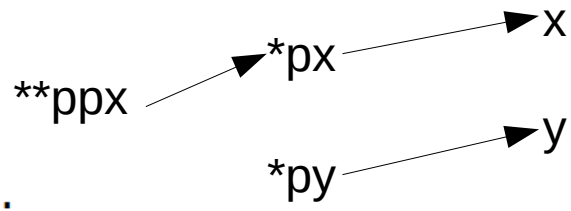
Ponteiro de Ponteiro **ppx

```
int x = 10, y = 20;
int *px, *py, **ppx;

px = &x;
py = &y;
ppx = &px;

printf("\n Valor de X = %d",x);
printf("\n Valor de *PX = %d",*px);
printf("\n Valor de *PPX = %d",**ppx);

ppx = &py;

printf("\n Conteudo referenciado *PX = %d",*px);
printf("\n Conteudo referenciado *PPX = %d\n",**ppx);
```

Troca no ponteiro *p causa troca no **r

```
Valor de X = 10
Valor de *PX = 10
Valor de *PPX = 10
Conteudo referenciado *PX = 20
Conteudo referenciado *PPX = 20
```

Execução algoritmo - Troca



Exemplo12-Ponteiro2.c

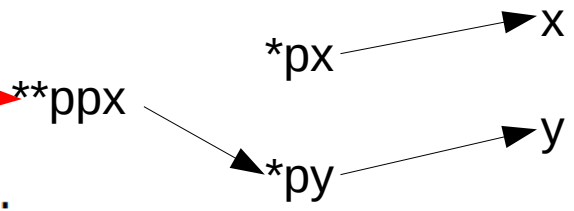
Ponteiro de Ponteiro **ppx

```
int x = 10, y = 20;
int *px, *py, **ppx;

px = &x;
py = &y;
ppx = &px;

printf("\n Valor de X = %d",x);
printf("\n Valor de *PX = %d",*px);
printf("\n Valor de *PPX = %d",**ppx);

ppx = &py;

printf("\n Conteudo referenciado *PX = %d",*px);
printf("\n Conteudo referenciado *PPX = %d\n",**ppx);
```

Troca no ponteiro *p causa troca no **r

```
Valor de X = 10
Valor de *PX = 10
Valor de *PPX = 10
Conteudo referenciado *PX = 20
Conteudo referenciado *PPX = 20
```

Execução algoritmo - Troca



Ponteiro de Ponteiro **ppx

```c
int x = 10, y = 20;
int *px, *py, **ppx;

px = &x;
py = &y;
ppx = &px;

printf("\n Valor de X = %d",x);
printf("\n Valor de *PX = %d",*px);
printf("\n Valor de *PPX = %d",**ppx);

*ppx = py;

printf("\n Conteudo referenciado *PX = %d",*px);
printf("\n Conteudo referenciado *PPX = %d\n",**ppx);

**ppx = 50;

printf("\n Valor de Y = %d",y);
printf("\n Valor de *PX = %d",*px);
printf("\n Valor de *PPX = %d\n",**ppx);
```

Troca no ponteiro *p causa troca no **r

```
Valor de X = 10
Valor de *PX = 10
Valor de *PPX = 10
Conteudo referenciado *PX = 20
Conteudo referenciado *PPX = 20

Valor de Y = 50
Valor de *PX = 50
Valor de *PPX = 50
```

Exercícios

17)Com base no exemplo anterior, ponteiro de ponteiro, troque o valor de X para 100, usando Ppx;

18)Crie um ponteiro de inteiro e aloque memória para um inteiro, armazene a soma de X e Y, usando os ponteiros;

19)Troque os endereços armazenados em px e py, mostre os valores armazenados X,Y,Px,Py,PPX